Hi Chat! I'm beginning an assignment task for a gaming company to show my skills as a Unity developer. They are fine with me using AI tools, so I want you to assist me while implementing the task. I will share our discussion with the team. I want you to be precise, clear and answer meaningfully and compactly. Don't send me code unless I ask you to. Don't generate too much text if it's not required. I need two things from you: - high-level proofs of my ideas - low-level implementations of specific parts of features (as much isolated as you can) I would prefer to stick with following plan:
1. I upload the text of the task
2. I describe the high-level structure
3. You give me feedback on my ideas and highlight bottlenecks
4. I implement features (you can help me with specific parts of the code)
5. I upload the code and you give me feedback on my code according the next principles:
- is any piece of code stable, errorless, solid, optimal etc
- is this code corresponds to modern code style, SOLID principles, flexible enough, can be re-used, etc
- is this code corresponding to what I was asked in the task.
6. You help me with creating documentation and a readme file for GitHub repository so this task is clear, can be easily understood, and corresponds to what the team expects from me.
Next step: I will upload the task.

==========

Sounds good. Upload the task text when ready.

I'll keep feedback compact and split it into:

1. what matches the assignment,
2. risks/bottlenecks,
3. architectural suggestions,
4. implementation-level notes only when needed.

I won't send code unless you ask for it.

==========

I've reviewed the task.

This is actually a very good assignment because they are evaluating:

- scoping decisions,
- architecture under time pressure,
- pragmatism,
- AI collaboration transparency,
- and code cleanliness more than feature complexity.

Main hidden requirement:
They explicitly want to see whether you avoid overengineering while still designing extensibly.

Key implications:

- Don’t build full Solitaire.
- Don’t build generic command history architecture for 20 systems.
- But also don’t hardcode everything into one MonoBehaviour.

A strong submission probably looks like:

- 2–3 card stacks,
- draggable cards,
- one-step undo,
- minimal but clean state abstraction,
- very readable folder structure,
- simple README,
- explicit explanation of AI usage.

Potential bottlenecks:

- overcomplicated drag system,
- coupling visuals with game state,
- undo restoring visuals incorrectly,
- trying to support “future scalability” too aggressively.

The safest architecture direction is probably:

- Card model/state
- Stack/container abstraction
- Move action data object
- Undo service/history manager
- Thin input layer
- Visual layer separated from move logic

That gives:

- modularity,
- SOLID signals,
- testability,
- while still remaining tiny.

Next step:
Describe your planned high-level structure and feature scope.

==========

Okay! I suggest starting with implementing a simple gameplay setup. The task doesn't require me to implement game logic and generate game field etc. So I would avoid going too deep in that direction. A thin game round controller is enough I suppose. Game controller is based on stacks. This data structure ideally fits the game rules. As we don't need to implement gameplay, but just turns by dragging and dropping cards we will have a Map of Stacks (so we can find a stack by Id). A game turn will be an operation of popping a card

from one stack and pushing to another. To be able to undo the turn we can go with two ways: - simple - keeping two stack Ids: last pop stack id and last push stack id, so we can pop a card from last push stack and push to last pop stack. It will be the Undo operation - extended - keeping history of operations. Each operation is a structure with stack ids (pop and push like in the simple one), and then we create a stack of operations, so we can undo as many turns as we have in the history stack. What do you think is a better fit for the task?

==========

The **extended way** is better, but keep it very small.

Reason: the task asks for **at least 1 previous move**, but they also evaluate architecture and critical thinking. A tiny move-history stack shows good design without much extra complexity.

Recommended scope:

MoveRecord
- CardId / Card reference
- FromStackId
- ToStackId
UndoService / MoveHistory
- PushMove(record)
- TryPopLastMove(out record)
- CanUndo

Then undo means:

take exact moved card from ToStack
return it to FromStack

Why not only store two stack IDs:

- It assumes the moved card is always on top of the destination stack.
- It breaks easily if later you support multi-card moves, invalid states, animations, or intermediate changes.
- It looks more like a shortcut than a deliberate design.

Why not overdo history:

- No need for redo.
- No need for command pattern unless you want to mention it as a future improvement.
- No need for serialization.

Best fit for this task:

GameRoundController
- owns stacks
- validates/moves cards

- records successful moves
- calls undo service
CardStack
- Push(card)
- Pop()
- Remove(card) maybe optional
- Peek()
MoveHistory
- Stack<MoveRecord>

This gives you a clean proof point: **one-step undo works now, multi-step undo is naturally supported, but feature remains scoped for 2 hours.**

==========

So let's start from creating a project. I will create a scene, I won't create any arts or something as I was not asked. I will use a 2D Unity setup.

==========

Good choice. For this task, **2D with placeholder sprites/UI is completely appropriate**.

I'd keep the first scene very minimal:

Scene
- Main Camera
- GameRoundController
- Canvas
 - Undo Button
- Stack_1
- Stack_2
- Stack_3
- A few Card objects

Use simple colored rectangles or Unity UI Images/Text labels. No custom art needed.

Suggested first implementation milestone:

Goal: manually placed cards can be moved between stacks.
No undo yet.
No real Solitaire rules yet.

Recommended project folders:

Assets/
 Scripts/
  Cards/

Stacks/
Gameplay/
Undo/
Input/
Scenes/
Prefabs/

For now, focus on proving:

- stacks exist,
- cards belong to stacks,
- a move is routed through the controller,
- visuals update after a move.

That gives a clean foundation before adding undo.

==========

Okay, sounds good! While I'm creating a project I want you to create a Drag and Drop system. This system should be simple, isolated and based on Unity API, so we can re-use it. It should have an event-based API. Like
interface IDraggableItem{ string ID; }
interface IDraggableDock{ string ID; }
class DragInfo { IDraggableItem Item; IDraggableDock From; IDraggableDock To; }
public class DragAndDropService { Action<DragInfo> OnItemDragged }
It's Just an Idea. If you think it should look different, you can do it. But keep in mind, it should be an encapsulated and isolated system not connected to the project code, so it can be easily re-used in other projects without changes (by including it as a package). Another thing, you should use Unity API, such as Unity events (OnDrag and OnDrop, Colliders etc), it's completely fine by the task.

==========

I'll keep this as a reusable mini-package: no Solitaire references, no game rules, and events only report drag intent/result. I'll base it on Unity's EventSystem interfaces, which is cleaner than manual collider polling for 2D/UI-style dragging.

Думал на протяжении 8s

Yes, this is a good isolated subsystem. I'd make one small change: **separate drag lifecycle events from successful drop events**.

Unity's EventSystem supports `IBeginDragHandler`, `IDragHandler`, `IEndDragHandler`, and `IDropHandler`, so this fits the engine API well.

Proposed reusable package:

DragAndDrop/

IDraggableItem.cs
IDraggableDock.cs
DragInfo.cs
DraggableItemView.cs
DraggableDockView.cs
DragAndDropService.cs

Core API:

```
public interface IDraggableItem
{
  string Id { get; }
}

public interface IDraggableDock
{
  string Id { get; }
}
public readonly struct DragInfo
{
  public readonly IDraggableItem Item;
  public readonly IDraggableDock From;
  public readonly IDraggableDock To;

  public DragInfo(IDraggableItem item, IDraggableDock from, IDraggableDock to)
  {
     Item = item;
     From = from;
     To = to;
  }
}
public sealed class DragAndDropService
{
  public event Action<DragInfo> DragStarted;
  public event Action<DragInfo> Dropped;
  public event Action<DragInfo> DragCanceled;
}
```

Important design choice:

DraggableItemView
- knows how to visually drag itself
- remembers source dock
- does not know game rules

DraggableDockView
- receives drop
- reports target dock

- does not validate Solitaire logic

GameRoundController
- subscribes to Dropped
- decides whether move is allowed
- updates stack data
- triggers visual correction if needed

One practical note: for Unity UI drag/drop, cards and docks should usually be under a `Canvas`, and the scene needs:

- `EventSystem`
- `GraphicRaycaster` on Canvas
- `CanvasGroup` on draggable item

If you use world-space sprites instead, add a `Physics2DRaycaster` to the camera and colliders to objects.

For this assignment, I'd recommend **UI-based cards on Canvas**. It is faster, cleaner, and easier to demo.

==========

We will go with 2D sprites, Physics2DRaycaster and 2d-colliders. Implement the service that starts dragging an Item, creates draggable info, stores FromDock, handles release, if mouse pointer/touch position above an other dock, we complete the drag, otherwise we cancel. Remember, we need to check if DockTo is NOT DockFrom.

==========

I'll implement it as a small reusable module using Unity pointer events plus 2D raycast lookup on release. The service will own drag state and emit events; item/dock components only identify themselves.

Думал на протяжении 11s

Below is a reusable **2D Physics drag-and-drop module**. It has no Solitaire-specific logic.

Unity setup requirement: use `EventSystem`, `Physics2DRaycaster` on camera, and `Collider2D` on draggable items/docks. Unity's pointer event interfaces are designed for this event flow.

`IDraggableItem.cs`

```
namespace DragAndDrop
{
  public interface IDraggableItem
  {
```

```
        string Id { get; }
    }
}
```

IDraggableDock.cs

```
namespace DragAndDrop
{
    public interface IDraggableDock
    {
        string Id { get; }
    }
}
```

DragInfo.cs

```
namespace DragAndDrop
{
    public readonly struct DragInfo
    {
        public readonly IDraggableItem Item;
        public readonly IDraggableDock From;
        public readonly IDraggableDock To;

        public DragInfo(IDraggableItem item, IDraggableDock from, IDraggableDock to)
        {
            Item = item;
            From = from;
            To = to;
        }
    }
}
```

DragAndDropService.cs

```
using System;

namespace DragAndDrop
{
    public sealed class DragAndDropService
    {
        public event Action<DragInfo> DragStarted;
        public event Action<DragInfo> DragCompleted;
        public event Action<DragInfo> DragCanceled;

        public void StartDrag(IDraggableItem item, IDraggableDock from)
        {
            DragStarted?.Invoke(new DragInfo(item, from, null));
        }
```

```
        public void CompleteDrag(IDraggableItem item, IDraggableDock from, IDraggableDock
to)
        {
            DragCompleted?.Invoke(new DragInfo(item, from, to));
        }

        public void CancelDrag(IDraggableItem item, IDraggableDock from)
        {
            DragCanceled?.Invoke(new DragInfo(item, from, null));
        }
    }
}
```

DragAndDropServiceProvider.cs

```
namespace DragAndDrop
{
    public static class DragAndDropServiceProvider
    {
        public static readonly DragAndDropService Service = new();
    }
}
```

DraggableDockView.cs

```
using UnityEngine;

namespace DragAndDrop
{
    [DisallowMultipleComponent]
    public sealed class DraggableDockView : MonoBehaviour, IDraggableDock
    {
        [SerializeField] private string id;

        public string Id => id;
    }
}
```

DraggableItemView.cs

```
using UnityEngine;
using UnityEngine.EventSystems;

namespace DragAndDrop
{
    [DisallowMultipleComponent]
    public sealed class DraggableItemView : MonoBehaviour, IDraggableItem,
        IBeginDragHandler, IDragHandler, IEndDragHandler
```

```
{
    [SerializeField] private string id;
    [SerializeField] private DraggableDockView currentDock;
    [SerializeField] private Camera raycastCamera;

    private Vector3 _startPosition;
    private Vector3 _pointerOffset;
    private IDraggableDock _fromDock;

    public string Id => id;
    public DraggableDockView CurrentDock => currentDock;

    private void Awake()
    {
        if (raycastCamera == null)
            raycastCamera = Camera.main;
    }

    public void SetCurrentDock(DraggableDockView dock)
    {
        currentDock = dock;
    }

    public void OnBeginDrag(PointerEventData eventData)
    {
        _startPosition = transform.position;
        _fromDock = currentDock;

        Vector3 pointerWorldPosition = GetPointerWorldPosition(eventData);
        _pointerOffset = transform.position - pointerWorldPosition;

        DragAndDropServiceProvider.Service.StartDrag(this, _fromDock);
    }

    public void OnDrag(PointerEventData eventData)
    {
        transform.position = GetPointerWorldPosition(eventData) + _pointerOffset;
    }

    public void OnEndDrag(PointerEventData eventData)
    {
        DraggableDockView targetDock = FindDockUnderPointer(eventData);

        bool hasValidTarget =
            targetDock != null &&
            _fromDock != null &&
            targetDock.Id != _fromDock.Id;
```

```
            if (hasValidTarget)
            {
                currentDock = targetDock;
                DragAndDropServiceProvider.Service.CompleteDrag(this, _fromDock, targetDock);
            }
            else
            {
                transform.position = _startPosition;
                DragAndDropServiceProvider.Service.CancelDrag(this, _fromDock);
            }

            _fromDock = null;
        }

        private Vector3 GetPointerWorldPosition(PointerEventData eventData)
        {
            Vector3 screenPosition = eventData.position;
            screenPosition.z = -raycastCamera.transform.position.z;

            Vector3 worldPosition = raycastCamera.ScreenToWorldPoint(screenPosition);
            worldPosition.z = transform.position.z;

            return worldPosition;
        }

        private static DraggableDockView FindDockUnderPointer(PointerEventData eventData)
        {
            foreach (RaycastResult result in eventData.hovered)
            {
                if (result.gameObject.TryGetComponent(out DraggableDockView dock))
                    return dock;

                dock = result.gameObject.GetComponentInParent<DraggableDockView>();
                if (dock != null)
                    return dock;
            }

            return null;
        }
    }
}
```

One important note: `eventData.hovered` may include the dragged card itself. Make sure dock colliders are reachable by raycast, and if needed put cards/docks on clear sorting/layers.

==========

Yes, It looks good. I added some improvements, like magnetic positioning, made the code a bit more package-like, but in general it's ok. Next step is to create a Bootstrap class that creates the GameRoundController. In the real game it would be a special game rounder builder, but as we work on a demo project without a game loop it can be a Bootstrap class that knows everything about game round entities to be able to create them.

==========

Agreed. For this prototype, a **Bootstrap** class is a good pragmatic choice.

Recommended responsibility split:

Bootstrap
- references scene objects
- creates runtime services/controllers
- wires dependencies
- initializes demo state

GameRoundController
- owns stack map
- executes move requests
- records successful moves
- performs undo

StackView / CardView
- scene-side representations
- expose IDs / references
- no decision-making

Bootstrap should be allowed to “know too much” because it is composition code. That is acceptable and often cleaner than hiding setup behind unnecessary abstractions.

Suggested Bootstrap fields:

```
[SerializeField] private CardStackView[] stackViews;
[SerializeField] private CardView[] cardViews;
[SerializeField] private Button undoButton;
```

Suggested creation flow:

Awake/Start
1. Create MoveHistory
2. Create GameRoundController
3. Register stacks
4. Register initial cards in stacks
5. Subscribe drag completed event
6. Subscribe undo button

7. Refresh visuals

Important design point:

The Bootstrap should subscribe drag events and forward them to the controller, or the controller can subscribe directly. I'd prefer:

Bootstrap wires:
DragAndDropService.OnCompleted += GameRoundController.TryMove
UndoButton.onClick += GameRoundController.Undo

This keeps `GameRoundController` unaware of Unity input/package details, which is cleaner.

So yes: **Bootstrap is the composition root**, and for this assignment that's a strong, explainable choice.

==========

You can give me whole bootstrap class code, it's pretty simple

==========

Here is a simple Bootstrap shape I'd use:

```
using DragAndDrop;
using UnityEngine;
using UnityEngine.UI;

namespace SolitaireDemo.Gameplay
{
  public sealed class GameBootstrap : MonoBehaviour
  {
    [Header("Scene References")]
    [SerializeField] private CardStackView[] stackViews;
    [SerializeField] private CardView[] cardViews;
    [SerializeField] private Button undoButton;

    private GameRoundController _gameRoundController;
    private MoveHistory _moveHistory;

    private void Awake()
    {
      _moveHistory = new MoveHistory();

      _gameRoundController = new GameRoundController(
        stackViews,
        cardViews,
```

```
            _moveHistory
        );

        DragAndDropServiceProvider.Service.DragCompleted += OnDragCompleted;

        if (undoButton != null)
            undoButton.onClick.AddListener(OnUndoClicked);
    }

    private void Start()
    {
        _gameRoundController.Initialize();
        RefreshUndoButton();
    }

    private void OnDestroy()
    {
        DragAndDropServiceProvider.Service.DragCompleted -= OnDragCompleted;

        if (undoButton != null)
            undoButton.onClick.RemoveListener(OnUndoClicked);
    }

    private void OnDragCompleted(DragInfo dragInfo)
    {
        _gameRoundController.TryMoveCard(
            dragInfo.Item.Id,
            dragInfo.From.Id,
            dragInfo.To.Id
        );

        RefreshUndoButton();
    }

    private void OnUndoClicked()
    {
        _gameRoundController.TryUndo();
        RefreshUndoButton();
    }

    private void RefreshUndoButton()
    {
        if (undoButton != null)
            undoButton.interactable = _gameRoundController.CanUndo;
    }
  }
}
```

This assumes:

CardView
- has Id

CardStackView
- has Id

GameRoundController
- Initialize()
- TryMoveCard(cardId, fromStackId, toStackId)
- TryUndo()
- CanUndo

The important part is that Bootstrap only wires dependencies and scene references. Actual move/undo logic stays outside it.

==========

No, actually I don't like how it works now. I want to separate responsibilities. Despite that we work on a Demo project I don't want Bootstrap to do everything. So I will create a Composition class that will create a GameRoundController and GameFieldView to control card views. And bootstrap will transfer mono behavior components into composition class. So the responsibility of Bootstrap is to provide objects from a scene and Initialize composition class. The responsibility of the composition class is to compose everything and bind all the dependencies. GameRoundController's responsibility is to react on drag events and run the game logic. GameFieldView's responsibility is to update game views.

==========

I like this direction better. It gives you a cleaner **composition root** without making Bootstrap a “god object”.

Suggested responsibility split:

GameBootstrap : MonoBehaviour
- serialized scene references only
- calls composition.Initialize(...)
- calls composition.Dispose() on destroy
GameComposition
- creates services/controllers
- wires dependencies/events
- owns lifetime of plain C# objects
GameRoundController
- subscribes/reacts to drag events
- validates and executes moves
- records undo history
- exposes events/state changes

GameFieldView
- owns visual card/stack mapping
- places cards
- restores card position after canceled/invalid move
- updates whole field from game state

This is stronger than the previous version because:

- Bootstrap stays Unity-specific and dumb.
- Composition is testable-ish plain C# setup code.
- Controller is not polluted with scene lookup.
- View updates are not mixed with game rules.

One recommendation: don't let `GameRoundController` directly move transforms. Let it emit something like:

MoveApplied
MoveRejected
UndoApplied

Then `GameFieldView` reacts and updates visuals.

Good final flow:

DragAndDropService
  ↓
GameRoundController
  ↓ emits state/result events
GameFieldView
  ↓
Card/Stack visuals

This is a better architecture signal for the assignment.

==========

I modified it a little bit and created some more task-specific classes, so I will upload the code for you to analyze it. Feel free to give me feedback as I mentioned in the beginning.

==========

Good architecture direction overall. The **Bootstrap → CompositionRoot → Controller/View** split is much better than putting setup logic into Bootstrap.

Main issues I see:

Critical / likely compile issues

1. **`DragAndDropService : MonoBehaviour` + `new()` singleton is wrong**

public sealed class DragAndDropService : MonoBehaviour
...
public static readonly DragAndDropService Service = new();

A `MonoBehaviour` should not be created with `new`. Make `DragAndDropService` a plain C# class, or place it on a GameObject and reference it from Bootstrap.

For your current design, plain C# class is better.

2. **`RaycastResult` usage looks incorrect**

In `DraggableItemView`:

result.TryGetComponent(...)
result.GetComponentInParent(...)

`RaycastResult` does not expose those directly. You probably need:

result.gameObject.TryGetComponent(...)
result.gameObject.GetComponentInParent(...)

Architecture feedback

Very good:

- `GameBootstrap` is now thin.
- `GameCompositionRoot` owns wiring.
- `GameRoundController` owns turn confirmation/undo.
- `GameFieldView` owns visual updates.
- `TurnInfo.Reverse()` is simple and readable.
- `TurnHistoryService` is isolated and easy to replace later.

Needs improvement:

- `GameTurnProcessor` implements `IDisposable`, but `GameCompositionRoot` does not dispose it. It stays subscribed to `DragAndDropService.DragRequest`.
- `GameFieldView` has its own serialized `_undoButton`, while `GameCompositionRoot` also receives `_undoButton`. This splits UI ownership. Prefer one owner.
- `GameRoundController` exposes events as public fields:

public Action<TurnInfo> OnTurnComplete;

Prefer events:

public event Action<TurnInfo> TurnCompleted;

Stability risks

1. `GameTurnProcessor.OnDragRequest` can throw:

return _turnHandler.Invoke(turnInfo);

If no handler is subscribed, this crashes. Safer to return `false`.

2. Dictionary access in `GameFieldView` can throw before your null checks:

var card = _cardViewDictionary[turnInfo.CardId];

Use `TryGetValue` if you want controlled error messages.

3. `_collider.enabled = false;` has no null check. Either require it with validation or auto-fetch it in `Awake`.
4. `TurnHistoryService` currently limits undo to 1 using `_undoCounter`, but since you pop the turn from history, the counter is probably unnecessary. For “only one undo” you can simply keep one record or clear history after undo.

Design note

I would remove `MagnetPoint` from `IDraggableDock`.

Right now the reusable drag-and-drop package knows about magnetic positioning:

Vector3 MagnetPoint { get; }

That makes it less generic. Better:

- drag/drop package: only detects item/from/to
- gameplay/view layer: decides where the card should snap

So `MagnetPoint` belongs to `StackView`, not `IDraggableDock`.

Overall

This is a solid direction and matches the assignment well. The biggest fixes before continuing:

1. make `DragAndDropService` not inherit `MonoBehaviour`,
2. fix `RaycastResult.gameObject` usage,
3. dispose `GameTurnProcessor`,
4. clean up undo button ownership,
5. move `MagnetPoint` out of the reusable drag/drop interface.

==========

1. Yes, that was made by mistake
2. New Unity API return Game Object as a result
3,4,5 I fixed them, you are right
Here's a new archive

==========

I'll compare the new version mainly against the previous risks: lifetime/disposal, package boundaries, undo flow, and scene/view ownership.

Думал на протяжении 13s

Much better. The main previous architectural issues are fixed.

Remaining feedback:

Good now

- `DragAndDropService` is now a plain C# class.
- `GameTurnProcessor` unsubscribes from drag events.
- `GameRoundController` now exposes proper events.
- `GameFieldView` owns UI/view updates.
- `Bootstrap` is thin enough.
- `GameCompositionRoot` is now a real composition layer.

Main issue left

`DraggableDockView` is `internal`, but `StackView` is `public` and exposes it in a serialized field:

[SerializeField] private DraggableDockView _draggableDockView;

This may compile inside the same assembly, but it makes package boundaries awkward. Either make `StackView` internal too, or make the drag/drop components public if they are meant to be reusable from outside the assembly.

For this demo, I'd make gameplay classes consistently `internal`.

Package-boundary issue

`IDraggableDock` is clean now, but `DraggableDockView` still contains:

public Vector3 MagnetPoint => ...

That is okay if `DraggableDockView` is not part of the reusable package contract. But conceptually, magnetic positioning is still not drag/drop responsibility.

Cleaner split:

DraggableDockView
- Id only

StackView
- Position / MagnetPoint

Possible runtime risks

1. `_collider.enabled = false` can throw if not assigned.

Add either:

- `[RequireComponent(typeof(Collider2D))]`, or
- auto-fetch in `Awake`.

2. `_undoButton` can throw if not assigned.

Not critical for demo, but easy to guard.

3. Dictionary `.Add()` can throw on duplicate IDs.

That is actually acceptable for this assignment, but a clearer validation log would be nicer.

Undo logic

This part works:

AddTurn -> Push
Undo -> Pop -> Reverse -> update view

But `_undoCounter` is unnecessary because after `Pop()`, `CanUndo` becomes false anyway for one move unless another move is added.

For single-step undo, simpler:

store one nullable TurnInfo

For multi-step undo, remove `_undoCounter`.

Current version is not wrong, just slightly confusing.

One compile concern

You said newer Unity API returns `GameObject`-like access on `RaycastResult`. If your Unity version supports this:

result.TryGetComponent(...)

then fine. If not, this is the first place I'd check.

Overall

This is now a solid prototype architecture. I'd only clean up:

1. consistent visibility,
2. collider/button null safety,
3. simplify `TurnHistoryService`,
4. move `MagnetPoint` fully into gameplay view layer.

==========

Main issues Fixed

Possible runtime risks 1 and 2. It should be assigned. For real projects we will use Odin for checking dependencies. I'm aware of this problem but I think it's fine by the task

3 Undo Counter should be if we want to undo multiple steps. It's easy to have, but give us flexibility and doesn't break logic

==========

Agreed.

For the README/code comments, I'd phrase it like this:

> `TurnHistoryService` currently supports configurable undo depth. For the prototype it can be limited to one previous move to match the task requirements, while the same implementation allows increasing the limit without changing game flow.

That explains the counter as a deliberate flexibility point, not accidental complexity.

For assigned dependencies, that's acceptable for a demo. I'd still maybe add `[SerializeField]` + clear field names, and optionally `[RequireComponent]` only where it costs nothing. But no need to over-polish.

You're in a good state to move to the actual gameplay setup / scene binding now.

==========

Everything is ready. I have tested the game. It works. I came up with only one test case: we move a card from one dock to another then undo, then move one more time, then undo again and see if it returns to a right position. It couldn't work because the Card can forget its from-dock because of undo and return onto a wrong position when second undo

==========

Good catch. This is exactly the kind of test case worth mentioning in the README.

Root cause:

Undo updates game/view state,
but the draggable item's internal current dock is not updated.

So after undo, the card visually returns, but its drag component still thinks its `FromDock` is the previous dock. Then the next move records wrong source data.

Correct fix:

When `GameFieldView` applies a move or undo, it should update **both**:

1. visual position
2. draggable current dock

So the source of truth stays synchronized.

Expected flow:

Move A -> B
- model says card is in B
- view snaps card to B
- draggable item currentDock = B

Undo B -> A
- model says card is back in A
- view snaps card to A
- draggable item currentDock = A

This is a good architectural lesson:

Drag state is part of interaction state, not only visual state.
Any system that repositions cards programmatically must also update the draggable dock reference.

I'd add one method to `CardView`, something like:

SetDock(StackView stackView)

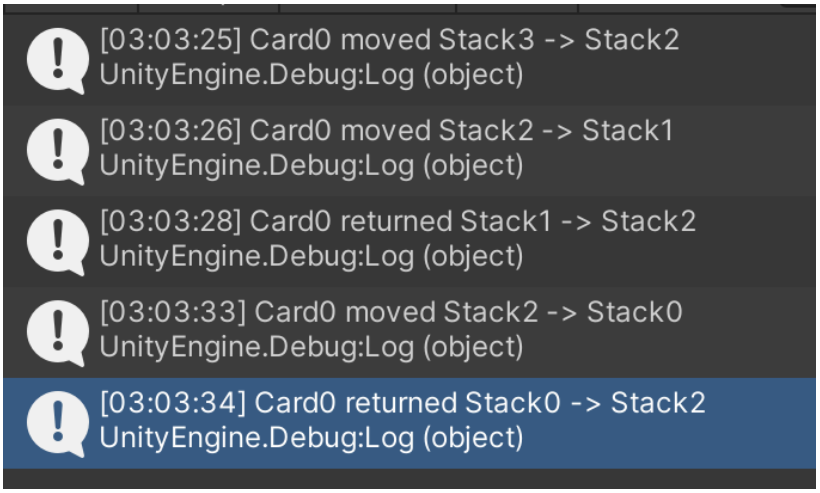
and inside it:

- update draggable dock reference,
- snap to stack/magnet point.

Then `GameFieldView` always calls that method instead of only moving transform position.

==========

Here's the result of the test under the test case I mentioned above. It looks good



==========

Yes, this log confirms the flow is correct now.

Sequence:

Stack3 -> Stack2
Stack2 -> Stack1
Undo: Stack1 -> Stack2
Stack2 -> Stack0
Undo: Stack0 -> Stack2

The important proof is the last line:

returned Stack0 -> Stack2

not

returned Stack0 -> Stack1

which means the draggable/source state was successfully synchronized after undo.

This is actually a strong test case because it validates:

- undo correctness,
- interaction state consistency,
- history integrity after state rollback.

I would absolutely mention this in README under “Testing / Edge Cases Considered”.

==========

So, I would say we are done with code and we can make the documentation and readme file.